

## INHERITANCE

### Aim:

To understand and implement inheritance concepts in Java.

### PRE LAB EXERCISE

#### QUESTIONS

- ✓ What is inheritance?

Inheritance is a feature in Java where one class (child class) gets the properties and methods of another class (parent class).

It helps create a relationship between classes and allows sharing of common features.

- ✓ What is code reusability?

Code reusability means using the same code again instead of writing it repeatedly.

In inheritance, the child class reuses the parent class methods and variables, which saves time and reduces duplication.

- ✓ What is the use of extends keyword?

The extends keyword is used to create inheritance in Java.

It allows a child class to inherit the properties and methods of a parent class.

### IN LAB EXERCISE

#### Objective:

To implement all types of inheritance.

#### PROGRAMS:

##### Student Result System (Single Inheritance)

#### Question:

A school wants to store student details and calculate marks. Create a base class Student and a derived class Result.

#### Code:

```
class Student {  
    String name;
```

```
int rollNo;

void getDetails() {
    name = "Anitha";
    rollNo = 101;
}
}

class Result extends Student {
    int marks = 85;

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Roll No: " + rollNo);
        System.out.println("Marks: " + marks);
    }
}

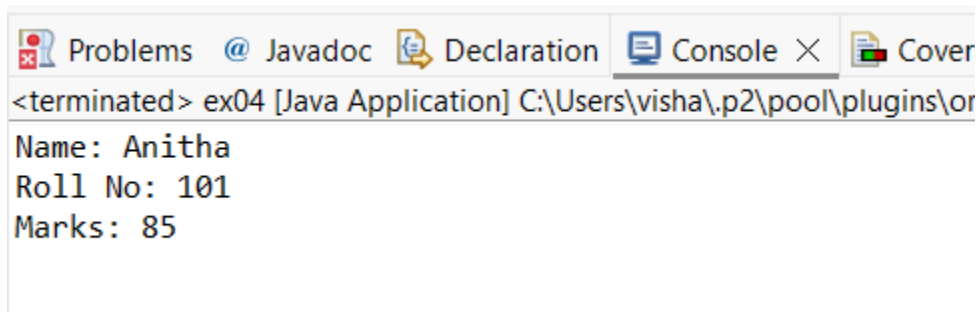
public class Main {
    public static void main(String[] args) {
        Result r = new Result();
        r.getDetails();
        r.display();
    }
}
```

**Output:**

Name: RAM

Roll No: 101

Marks: 85

A screenshot of an IDE's console window. The title bar shows tabs for 'Problems', 'Javadoc', 'Declaration', 'Console', and 'Cover'. The console text shows the program has terminated and displays the following output:

```
<terminated> ex04 [Java Application] C:\Users\visha\.p2\pool\plugins\or
Name: Anitha
Roll No: 101
Marks: 85
```

## 2. Bank Account System (Hierarchical Inheritance)

### Question:

A bank has Savings and Current accounts. Both inherit from a common Account class.

### Code:

```
class Account {
    void showAccountType() {
        System.out.println("Bank Account");
    }
}

class SavingsAccount extends Account {
    void interest() {
        System.out.println("Savings Account gives interest");
    }
}

class CurrentAccount extends Account {
    void overdraft() {
        System.out.println("Current Account supports overdraft");
    }
}
```

```

public class Main {
    public static void main(String[] args) {
        SavingsAccount s = new SavingsAccount();
        CurrentAccount c = new CurrentAccount();

        s.showAccountType();
        s.interest();

        c.showAccountType();
        c.overdraft();
    }
}

```

**Output:**

Bank Account

Savings Account gives interest

Bank Account

Current Account supports overdraft

```

Bank Account
Savings Account gives interest
Bank Account
Current Account supports overdraft

```

### 3. Vehicle System (Multilevel Inheritance)

**Question:**

A company classifies vehicles as Vehicle → Car → ElectricCar.

**Code:**

```

class Vehicle {

```

```
void start() {  
    System.out.println("Vehicle starts");  
}  
}  
  
class Car extends Vehicle {  
    void fuelType() {  
        System.out.println("Car uses petrol");  
    }  
}  
  
class ElectricCar extends Car {  
    void battery() {  
        System.out.println("Electric car uses battery");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        ElectricCar e = new ElectricCar();  
        e.start();  
        e.fuelType();  
        e.battery();  
    }  
}
```

**Output:**

Vehicle starts

Car uses petrol

Electric car uses battery

```
Vehicle starts  
Car uses petrol  
Electric car uses battery
```

## POST LAB EXERCISE

- ✓ Why Java does not support multiple inheritance using classes and how it is implemented?

- ✓ **Reason:**

Java does not support multiple inheritance using classes to avoid ambiguity problem (Diamond problem).

- Example problem:**

If two parent classes have the same method and a child inherits both, Java will not know which method to use.

- How it is implemented:**

Java supports multiple inheritance using interfaces.

- ✓ What is the role of the super keyword? Give examples.

The super keyword is used to refer to the parent (super) class from a child class.

Call parent class method

When a child class has a method with the same name as the parent class, the child's method overrides the parent's method.

If we want to use the parent class version of the method, we use the super keyword.

Purpose:

- To avoid confusion between parent and child methods
- To reuse the original functionality of the parent class
- To access the parent's implementation when it is overridden in the child

- ✓ Can a child class access private members of the parent class? Why?

**No.**

**Reason:**

Private members are accessible only within the same class.

They are not visible to child classes.

- ✓ Explain why hybrid inheritance is not supported in Java.
- Hybrid inheritance = Combination of multiple inheritance and other types.
  - Java does not support it using classes because:
  - It may create multiple inheritance
  - Leads to ambiguity (diamond problem)
  - Makes code complex
  - But hybrid inheritance is possible using interfaces.

**Result:**

Thus the different types of inheritance were implemented and executed successfully.

**ASSESSMENT**

| Description       | Max Marks | Marks Awarded |
|-------------------|-----------|---------------|
| Pre Lab Exercise  | 5         |               |
| In Lab Exercise   | 10        |               |
| Post Lab Exercise | 5         |               |
| Viva              | 10        |               |
| Total             | 30        |               |
| Faculty Signature |           |               |